Intelligent Cyber-Attack Detection System

A machine learning pipeline designed to identify cyber-attacks within noisy network traffic with high precision.

alamy
Image ID: 3BM73TT
www.alamy.com

Project Overview

Objective

Build a robust classifier capable of categorizing network traffic into over 1,500 distinct labels with high precision.

Challenge

Address noisy real-world data containing typos, incorrect labels, and extreme outliers that plague traditional IDS systems.

Scale

Process 1.7 million network traffic samples with 84 statistical features representing flow characteristics.

Dataset Specifications

File Details

Filename: combined_noisy.csv

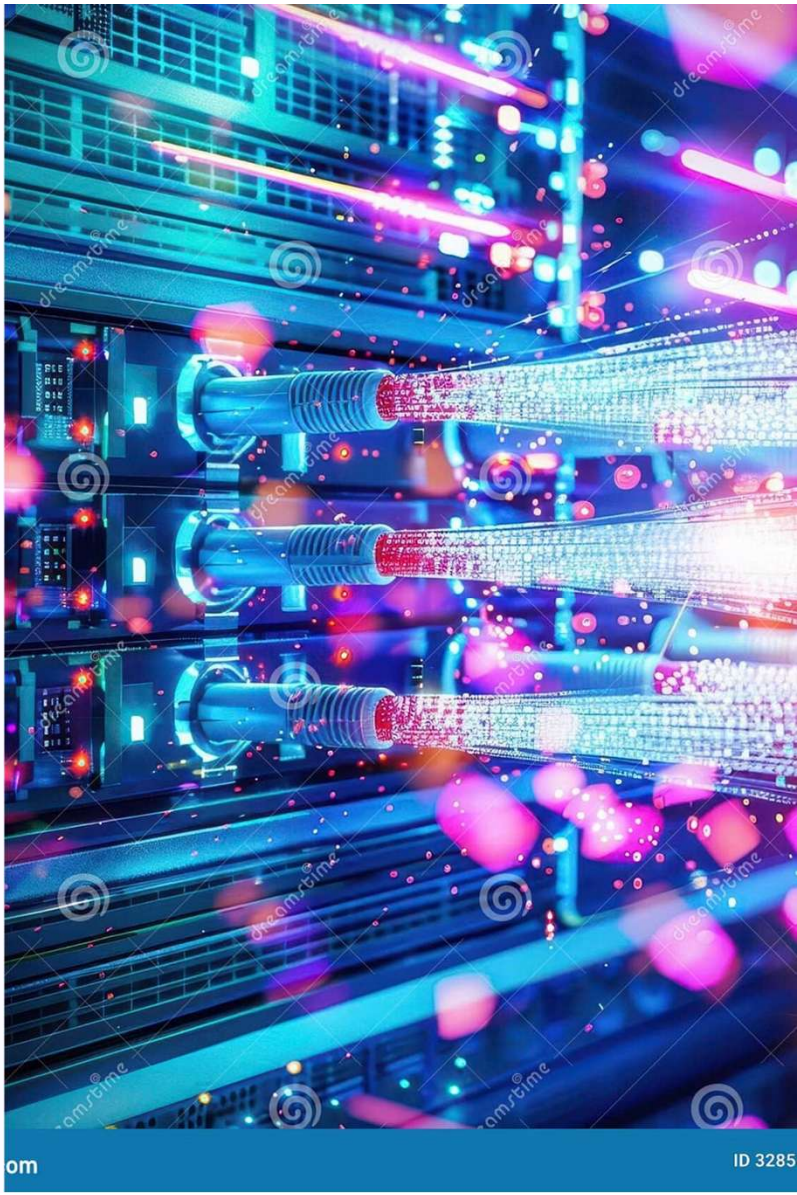
Samples: 1,775,435 network flows

Features: 84 columns representing network flow statistics including packet counts, flow duration, and inter-arrival times

Target: Label column with 1,588 unique classes representing benign traffic and various cyber-attacks

Attack Types

- Denial of Service (DoS)
- Port scanning attacks
- Distributed DoS (DDoS)
- Benign traffic patterns



Technical Architecture

The project is structured into three primary modules, each handled in a dedicated environment to ensure optimal performance and reproducibility.

Exploratory Data Analysis

Statistical profiling and anomaly detection to identify data quality issues.

Feature Engineering

Data cleaning, scaling, and transformation for model readiness.

Model Training

Training gradient-boosted decision trees on GPU hardware.

Phase 1: Exploratory Data Analysis

Noise Identification

Discovered that the high number of classes (1,588) was due to label noise—multiple spellings for the same attack type such as "PortScan" and "PcortScan" appearing as separate categories.

Data Integrity Check

Identified negative values in critical features like Flow Duration and Source Port, which were flagged as data collection errors requiring correction.

Outlier Analysis

Found that approximately 26% of features contained significant outliers, necessitating robust scaling approaches to prevent model bias.

Phase 2: Feature Engineering

Comprehensive data transformations were applied to prepare raw network traffic data for machine learning models.

01

Metadata Removal

Dropped non-predictive columns including Flow ID, Source IP, Destination IP, and Timestamp to ensure the model learns traffic patterns rather than memorizing specific addresses.

02

Handling Incompleteness

Removed 7,304 duplicate rows, replaced infinite values with NaN, and used SimpleImputer to fill missing values with median statistics.

03

Feature Scaling

Applied RobustScaler using interquartile range (IQR) to scale data, making it resilient to extreme outliers identified during EDA.

04

Label Encoding

Converted categorical text labels into integer format for compatibility with the LightGBM algorithm.

Phase 3: Model Development

Algorithm Selection

LightGBM (Gradient Boosting Machine) was chosen for its efficiency with large datasets and ability to handle high-dimensional sparse data.

Hardware Acceleration

Utilized NVIDIA Tesla T4 GPU to handle the large-scale dataset containing 1.7 million+ rows efficiently.

Validation Strategy

Stratified K-Fold cross-validation ensured that even the rarest attack classes were represented in both training and validation sets, preventing bias toward common attack types.

Results & Evaluation

86%

Overall Accuracy

Model correctly classified network traffic patterns

1.7M

Samples Processed

Total network flows analyzed

1,588

Attack Classes

Distinct label categories identified

Key Findings

Strong Performance on Common Attacks

The model performed exceptionally well on "Benign" traffic and common attack types, achieving F1-scores of 0.98+ for these categories.

Challenges with Noisy Labels

Performance dropped on highly noisy or "typo" labels with very few samples, indicating that further label consolidation would improve system accuracy.

Future Enhancements

The current pipeline successfully handles massive data volume and high-dimensional noise. Future iterations will focus on three key improvements:

Label Grouping

Implement fuzzy string matching algorithms to group noisy labels, merging all "DoS" variants and similar attack names into unified parent classes.

Class Imbalance

Apply SMOTE (Synthetic Minority Over-sampling Technique) to generate synthetic samples for rare attack types, improving detection rates for uncommon threats.

Real-time Integration

Export the trained model as a .pkl or .onnx file for integration into live network monitoring tools and production environments.

Tools & Libraries

Data Manipulation

pandas and numpy for efficient data handling and numerical computing operations.

Visualization

matplotlib and seaborn for creating statistical graphics and exploratory data visualizations.

Machine Learning

scikit-learn for preprocessing utilities and LightGBM for gradient boosting implementation.

Environment

Kaggle/Jupyter Notebook running Python 3.12 for reproducible research workflows.